CSIS-225 Final Semester Project Report

Team Number: 3
Team Name: Software Security
Team Members: Isaiah Santamaria, Hayden Ralston, Sean Powers
Application Title: Personal Finance Manager

Instructions for Running the Application

1. Clone the Repo(**do not** download **zip file version** because there will be bugs)
2. Play FinanceManager.java
    a. Option 1: Recommend
        i. Use the play button on FinanceManager(must need java extension in VS code to do this)
    b. Option 2:
        i. javac *.java
        ii. java FinanceManager.

Describe how the project reflects a meaningful object-oriented design, making effective use of interfaces, abstract classes, inheritance, etc. (400 - 500 words). If your project has a reasonable number of classes, including a UML diagram here would be helpful, but is not required.

In Our Program, we have java classes/files separated by folders in the repo which are backend, components, and db. FinanceManager.java utilizes all the classes

Components
Each panel that is displayed in the application is separated into mini components which are combined in Root.java. Main components are Banner.java, Intro.java, AccountsCont.java, RecentTransactionCont.java, GraphChart.java, and Input.java

Intro.java → is a thread classes that first welcomes the user before display Root.java
Root.java → JPanel that displays all other components into 1 JPanel while providing pointer objects so each component can communicate with one another
AccountsCont.java → displays total amount of users bank account, it displays Checking, Savings and Joint
Banner.java → displays the name of the software and its credentials
RecentTranstionCont.java → The application displays the highest number of transactions in a JTable and includes a print button. It also features the GraphChart.java module, which presents a chart comparing income and expenses.
Input.java → is the most complicated because it will first ask the user if they would like to input an expense or an input and from there, the whole panel would change to ask for a specific answer based on which answer they have chosen.

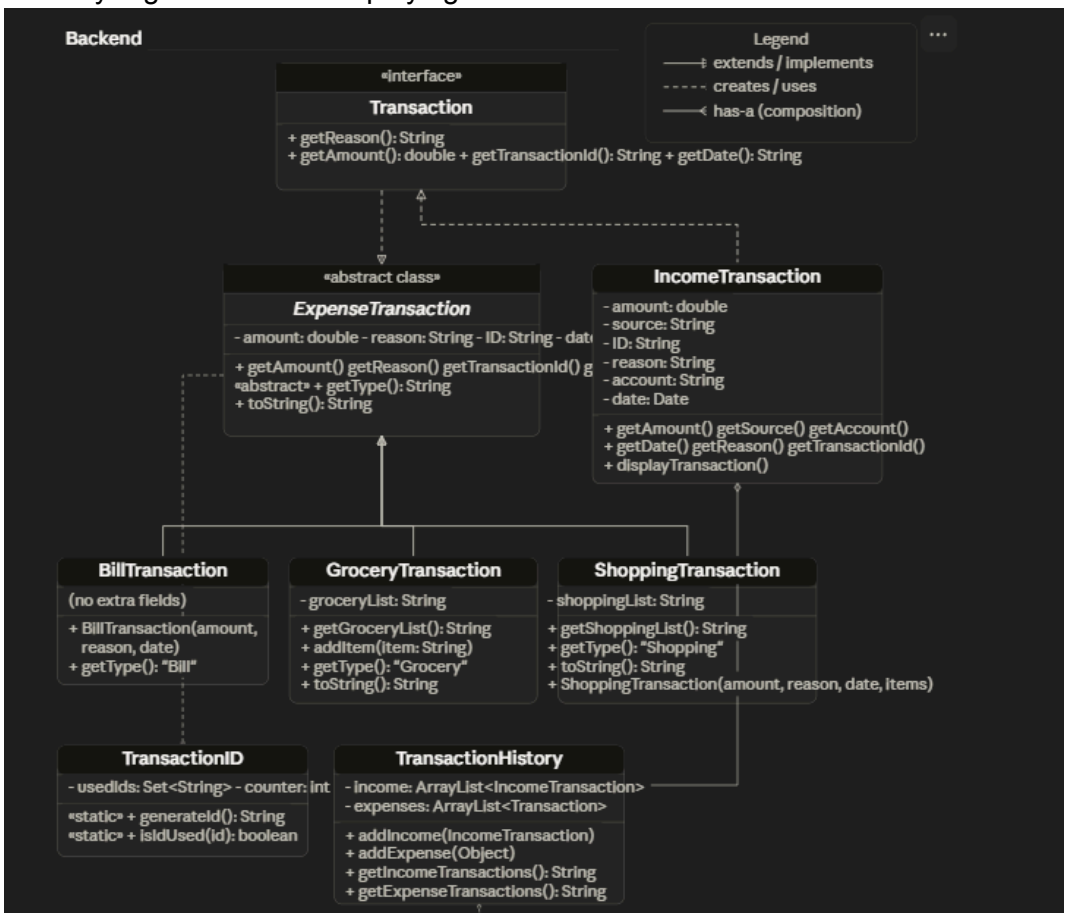
Backend:
This class is responsible for processing users' answers by turning them into transaction Objects and sending them to the DB folder.
ExpenseTransaction is used to establish the base of the transactions. The other transaction classes, Bill, Grocery, income, and shopping transactions all extend ExpenseTransaction. They all define their return methods so that the front end can access them. In addition, there is the csvTranslation and the EveryTransaction.csv files in order to keep track of the transactions themselves. These files keep track of all of the transactions so that they can be referenced.

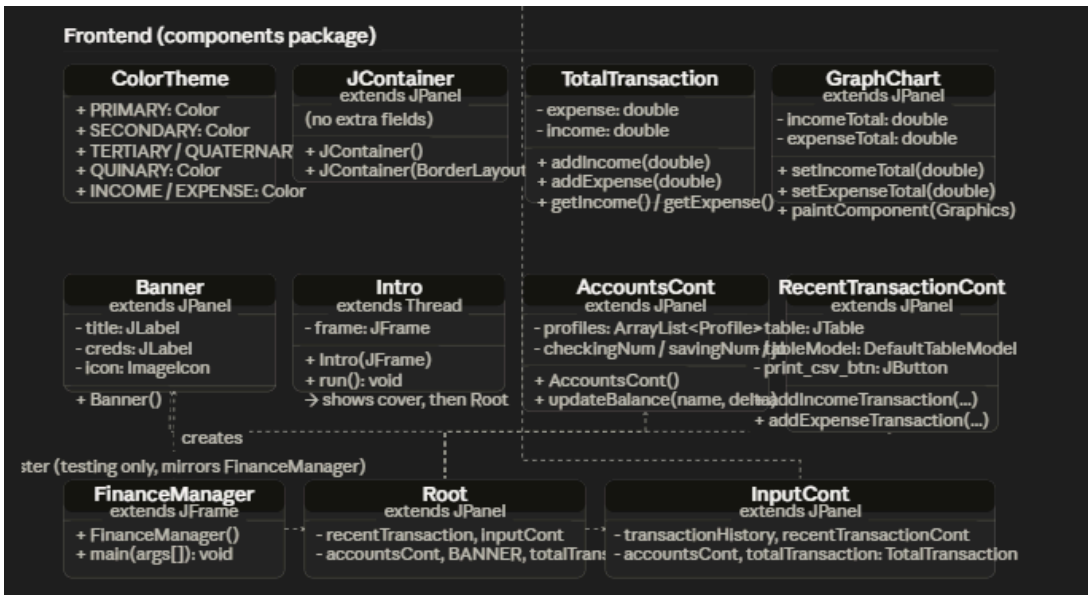
DB:
In the DB folder we used csv files to keep track of each transaction and its content. This helps keeps all of

the information even after the application has been closed

- database.csv that stores all the transactions that have been imputed.
- Income.csv keeps track of all income transactions
- expense.csv transaction stores all expense transactions.

The reason why we have multiple csv files instead of just 1 is because each transaction object has different content. By having multiple csv files we can display all of the content that each object contains so everything can stay organized while displaying all of the information each user needs.





List one class that implements event driven programming including Java graphics and Java Swing components.

InputCont.java in components folder contains event driven programming including Java graphics and Java Swing components.

List one class that implements threads. Then, describe the purpose and lifecycle of each treaded object (50 - 100 words).

The class Intro.java extends Thread. Its purpose is to display a temporary welcome screen without blocking the main GUI thread. The lifecycle begins when the intro instance is created and start() is called which triggers the run() method. This builds and shows a panel that says Welcome on a swing event thread which slowly fades to black and finally disappears into the background. Once this is completed it is replaced with the main panel that holds the application. Afterwards the thread completes it's execution and the sequence terminates

List up to 3 different data structures that have significant uses in the application and the class(es) in which they are used. Then, describe why you chose to use each (50 - 100 words per data structure).

Arrays - arrays are found within the InputCont class. They are utilized as instance variables to hold the different categories for the options of the input. This allows us to have our specified number of dropdown options for the selection types that we want without taking up too much space in memory.
ArraysList - This was used in csvTranslation, and TransactionHistory. It was used in order to dynamically add to the list. Our project allows the user to add a list of their expenses and income. However, there is no limit for the user so we used ArraysList in order to have a non-limited way to add those expenses and incomes, and to add it into the csvTranslation.
HashSets - This was found within the TransasctionID class. We utilized the hash sets for the different ids, making sure that there are different IDs used for the transactions. The hashsets allow us to assign each individual transaction a unique number so that they can be pulled and looked at, and see who made the transaction to store it. The transactions get stored in the same file even if there is a new instance of the program, so the ID helps us to keep track of the transaction.

Describe the Java or general programming feature or construct included in your project that was not specifically studied in this class or prerequisite classes. Include the class(es) in which this feature is implemented. (200 - 300 words)

The java feature that we implemented into our program is the *import javax.swing.table.DefaultTableModel.* We implemented this feature in our *root.java* file utilizing it to display our transaction data in an organized, structured tabular format. Its primary purpose is to output rows of income and expense data, so the users can very easily read and track their activity. The income and expense data is not stored within the table, the data is stored within DefaultTableModel which the table relies on to store the data to be output inside the table. This is shared amongst multiple components allowing for real time updates each time an income or expense is recorded. When a user inputs a new transaction through InputCont, which is then added into the tableModel. Because the table is bound to this model, any information updates automatically appear in the UI without the need for the user to manually refresh the table. The table's lifecycle and its model are reliant on the root file to be initialized. The table lasts for the entire lifetime of the application and continuously updates itself as new data is added and imputed. There is no separate thread needed as all updates occur in the swing event dispatch thread.

Describe how your program focuses on social good to create positive societal, environmental, or community impact (100 - 300 words).

Our project is designed to help users understand their personal finances and get a better picture of how to use the best practices to stay financially aware in their life. This project allows people to input their purchases and really look at why they used it, as well as input the money that they saved or

earned. It visualizes their finances and allows them to truly see what their habits are like. This helps to develop financial awareness as well as showing good financial habits. This allows the community the opportunity to reflect on where their money goes and take charge of their financial life.